CASE STUDY · FULL STACK GELİŞTİRİCİ · KASIM 2026

Ekip Görev Takip Uygulaması

Rüzgar santralindeki saha bakım ekibi için küçük bir web uygulaması

01 Özet

Senden küçük bir görev takip uygulaması yazmanı istiyoruz. Süren **07.11.2026 13:00**'te başlıyor, **09.11.2026 01:00**'de bitiyor; bu sürede yaklaşık 6–8 saatlik bir çalışma yeterli. Uygulamanın backend'ini, arayüzünü ve README dosyasını sen yazacak, uygulamayı da deploy edeceksin. Kullanacağın veri seti bu dokümanla birlikte geliyor.

SÜRE	HEDEF EFOR	TESLİM	SONRASI
07.11.2026 13:00 – 09.11.2026 01:00	6–8 saat; daha fazlasını harcamanı beklemiyoruz	Git repo linki ve deploy linki	30 dakikalık bir teslim görüşmesi

Her şeyi bitirmen şart değil. Neyi neden öne aldığını görmek, tüm maddelerin tamamlanmasından daha önemli.

02 Senaryo

Bir rüzgar santralinde 8 kişilik bir saha bakım ekibi çalışıyor. Ekip işlerini şu an elle tutulan bir tabloda takip ediyor. Bu tabloda kimin hangi işle uğraştığını, hangi işin geciktiğini ve geçen hafta neler olduğunu görmek zor.

Ekip lideri bu tablonun yerine geçecek küçük bir web uygulaması istiyor. Uygulamada görevler oluşturulup atanabilmeli, durumları takip edilebilmeli ve her değişiklik kayıt altına alınmalı. Ekibin bugüne kadar tuttuğu tablo, veri setinde.

03 İstenenler

Backend

- Her görevin şu alanları olsun: başlık, açıklama, durum (Yapılacak / Devam ediyor / Bitti), öncelik, atanan kişi ve son tarih.
- Görevler için CRUD endpoint'leri olsun: oluşturma, listeleme, güncelleme ve silme.
- Liste duruma, atanan kişiye ve geciken görevlere göre filtrelenebilsin ve sıralanabilsin.
- Her durum ve atama değişikliği kaydedilsin: kim, ne zaman, neyi değiştirdi. Bu kayıt, görevin değişiklik geçmişi (audit log) olarak okunabilsin.
- Hatalı request'lerde anlamlı bir hata mesajı ve doğru HTTP status code dönsün. Örneğin başlık boş olabilir ya da görev, sistemde olmayan bir kişiye atanmak istenebilir.
- Dil ve framework seçimi serbest. Biz Python/FastAPI ve PostgreSQL kullanıyoruz. SQLite da yeterli.

Frontend

- Görevleri liste ya da kanban board olarak göster; seçim senin.
- Arayüzden görev oluşturulabilsin, düzenlenebilsin ve bir kişiye atanabilsin; atanan kişi sonradan değiştirilebilsin. Bunun için bir form olsun, validation mesajları görünsün.
- Duruma, kişiye ve geciken görevlere göre filtreleme yapılabilsin.
- Görev detayında değişiklik geçmişi görünsün.
- Loading, empty ve error state'leri ele alınsın.
- React tercih ediliyor (biz Next.js kullanıyoruz). Tasarım sade olabilir, piksel mükemmelliği aramıyoruz.

Veri seti

- `dataset/` klasöründe üç CSV dosyası var: `users.csv` (9 kişi), `tasks.csv` (42 görev) ve `task_history.csv` (55 değişiklik kaydı). Sütunlar `dataset/README.md` dosyasında anlatılıyor.
- Uygulama ilk açıldığında bu veriyle dolu gelsin.
- Tablo elle tutulduğu için verinin temiz olduğunu varsayma.

README

- Uygulama tek komutla ya da en çok üç adımda çalışmalı. Bu adımları README'de yaz.
- Aldığın kararları ve varsayımları yaz. Örneğin bir görev ne zaman gecikmiş sayılır? Bitmiş bir görev yeniden açılabilir mi?
- Veri setinde karşılaştığın sorunları ve her biri için verdiğin kararı yaz.
- Bilerek yapmadıklarını ve 2 gün daha olsa neler ekleyeceğini yaz.
- Yaklaşık kaç saat çalıştığını ve yapay zekâ araçlarını nerede kullandığını belirt.

Deploy

- Uygulamayı public bir URL'e deploy et. Frontend ve backend aynı platformda ya da ayrı platformlarda olabilir.
- Ücretsiz katmanı olan bir platform yeterli; örneğin Vercel, Render ya da Netlify, veritabanı için Neon ya da Supabase. Para harcamanı beklemiyoruz.
- Deploy edilen uygulama veri setiyle dolu açılsın.
- Deploy linki teslim görüşmesine kadar çalışır kalsın; görüşmede uygulamayı bu linkten göstereceksin. Ücretsiz katmanda ilk açılış yavaşsa bunu README'de belirt.
- Database URL ve API key gibi secret'lar platformun environment variable'larında dursun, repo'da değil.
- AI botunu yaptıysan production'da açık bırakmak zorunda değilsin; local'de çalışması yeterli, görüşmede botu kodundan anlatırsın. Açık bırakırsan, giriş olmayan bir uygulamada botu herkesin kullanabileceğini düşün.
- Deploy linkini ve nasıl deploy ettiğini README'de kısaca yaz.

Kapsam dışı

Authentication (kişiler sabit bir listeden seçilebilir; deploy edilen uygulamanın login olmadan açık olması sorun değil), mobil uygulama, bildirimler ve drag & drop beklemiyoruz. Geniş bir test kapsamı da gerekmiyor; kritik yerlerde birkaç anlamlı test yeterli.

04 İsteğe bağlı: AI özet botu

İSTEĞE BAĞLI

Bu bölüm zorunlu değil; yapmadığın için puan kaybetmezsin. Ana kısım bittikten sonra vaktin kalırsa ekle.

- Bir kişi ya da tarih aralığı seçildiğinde ekibin aktivitesini birkaç cümlelik Türkçe bir özetle anlatan bir bot ekle. Örnek bir soru: "Bu hafta ne bitti, ne gecikti, ne takıldı?"
- Özetteki sayılar ve görev listesi backend'den gelsin. Model yalnızca kendisine verilen veriyi anlatsın. Özette geçen her görev, uygulamadaki kaydına bağlansın.
- Seçilen aralıkta hiç aktivite yoksa bot bunu söylesin, bir şey uydurmasın.
- Hangi modeli kullanacağın serbest, local bir model de olur. API key tanımlı değilse uygulamanın geri kalanı yine çalışsın; uygulamayı biz API key olmadan da çalıştırabilmeliyiz.

05 Çalışma kuralları

Git ve commit düzeni

- Repo'yu işe başlarken oluştur ve ilk commit'i hemen at. Boş bir boilerplate de olur.
- Süreç boyunca düzenli ve küçük commit'ler at. Her anlamlı adım ayrı bir commit olsun; örneğin dataset import'u, görev endpoint'leri, liste ekranı.
- Commit mesajı ne yaptığını anlatsın. "Görev listesine durum filtresi eklendi" iyi bir örnek; "düzeltme" ya da "wip" değil.
- Tüm işi tek bir commit'le teslim etme.
- Geçmişi yeniden yazma: squash, rebase ya da force push yapma. Commit geçmişini de değerlendiriyoruz.
- Teslim zamanından sonra repo'ya commit push etme. Sonradan fark ettiğin bir şey olursa görüşmede anlatırsın.

Secret'lar

- API key, şifre ya da `.env` dosyası repo'ya girmesin. Gereken environment variable'ları `.env.example` dosyasında örnek değerlerle göster.

Yapay zekâ ve hazır kod

- Yapay zekâ araçları (Copilot, ChatGPT, Claude vb.) serbest. Nerede kullandığını README'de kısaca yaz.
- Teslim görüşmesinde kodunun her parçasını açıklayabilmen gerekiyor.
- Framework, library ve scaffolding araçları serbest; örneğin `create-next-app`, UI bileşen kütüphaneleri ya da ORM'ler. Hazır bir görev takip projesini kopyalayıp teslim etmek ise serbest değil.

Sorular ve varsayımlar

- Case'i tek başına yap; başka birinden yardım alma.
- Belirsiz bir nokta görürsen makul bir varsayım yap, README'ye yaz ve devam et. Bizden cevap beklemeni istemiyoruz.

Süre

- Hedef efor 6–8 saat. Bu süreyi aşmaya başladıysan dur ve kalan işleri README'de "yapılmadı" olarak yaz.

06 Teslim ve sonrası

Teslim

- En geç **09.11.2026 01:00**'e kadar Git repo linkini ve deploy linkini WhatsApp üzerinden **0505 117 21 81** numarasına gönder.
- Repo private ise şu kullanıcıyı collaborator olarak ekle: `ahmet-yilmaz-cs`.
- Repo'da kaynak kod, README, `.env.example` ve dataset'i import eden seed script bulunsun.
- Teslim etmeden önce repo'yu boş bir klasöre clone'la ve README'deki adımlarla çalıştırmayı dene.

Teslim görüşmesi (30 dk)

Teslimden sonraki birkaç gün içinde seninle 30 dakikalık bir görüşme yapacağız. Görüşme için hazırlık yapman gerekmiyor. Uygulamanı local'den değil, deploy linkinden göstereceksin. Kodu da repo'dan açacağız. Ekranını paylaşabilmen yeterli.

1. Uygulamanı deploy linkinden açıp bize gösterirsin (5 dk).
2. Kodunun birkaç bölümünü repo'dan açıp bize anlatırsın (10 dk).
3. Aldığın kararlar üzerine konuşuruz (10 dk).
4. Senin sorularını konuşuruz (5 dk).

07 Neye bakacağız

Özellik sayısına ya da tasarımın güzelliğine değil, problemi nasıl düşündüğüne bakıyoruz.

Problemi anlama ve varsayımlar
Belirsiz yerleri fark etmen, gerekçeli kararlar vermen ve bunları yazman.

Veri modeli ve iş kuralları
Görevleri ve değişiklik geçmişini nasıl modellediğin. Gecikme, durum geçişi ve silme gibi kuralları nerede ve nasıl uyguladığın.

Backend
Anlaşılır endpoint'ler, anlamlı hata mesajları ve veri büyüdüğünde ne olacağını düşünmüş olman.

Frontend
Kullanışlı ekranlar; loading, empty ve error state'lerinin ele alınması.

Deploy
Uygulamanın deploy linkinde veriyle çalışması ve secret'ların doğru yönetilmesi.

Kod ve commit düzeni
Okunur kod, küçük ve anlamlı commit'ler, kritik yerlerde test.

Teslim görüşmesi
Kararlarını açıklaman, kodunda rahatça yol bulman ve eksiklerini kendin söylemen.

Kolay gelsin. Seninle görüşmeyi dört gözle bekliyoruz.